# in-class 5

Submit Assignment

**Due** Monday by 11:59pm **Points** 100 **Submitting** a file upload **File Types** py
**Available** until Dec 15 at 11:59pm

**ENGI E1006 - Intro to Computing for Engineers & Applied Scientists**

**Due: 11:59pm on Monday, December 9**
**Total Points: 100**

**Preliminaries:**

The goal of this homework is to do some very simple image processing tasks, utilizing numpy to hold the image data, and to allow us to easily modify the image data.

You will use *matplotlib* and *numpy*. Download the file **inclass5.py**, which already contains the necessary imports.

You will use the `plt.imshow(img)` method in matplotlib to display images. **It is important for this problem set to understand the structure of a numpy image.** .

Take a look at the function `rgb_ints_example`, which should return a 150x150 image pattern.

Calling `plt.imshow(rgb_ints_example())` will display that image.
You may find using the `imageshow(...)` function defined in the file gives you a better image display on some systems (don't worry much about how it works). The function can be used as follows:
`imshow(rgb_ints_example())`

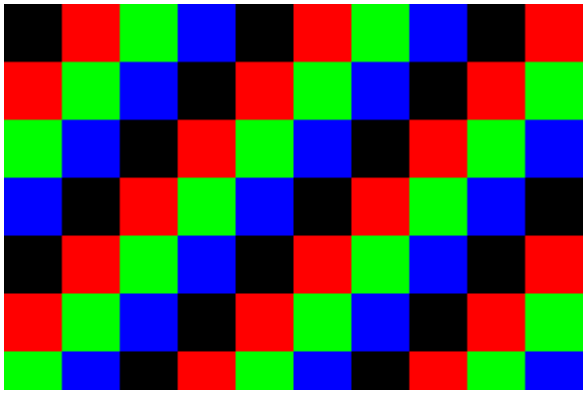
**Loading and displaying an image**

You can load an image using the `plt.imread(...)` method.

running:

```
pattern = plt.imread('pattern.png')
imageshow(pattern)
```

should display:



### Part 1: Color Channels (30 points)

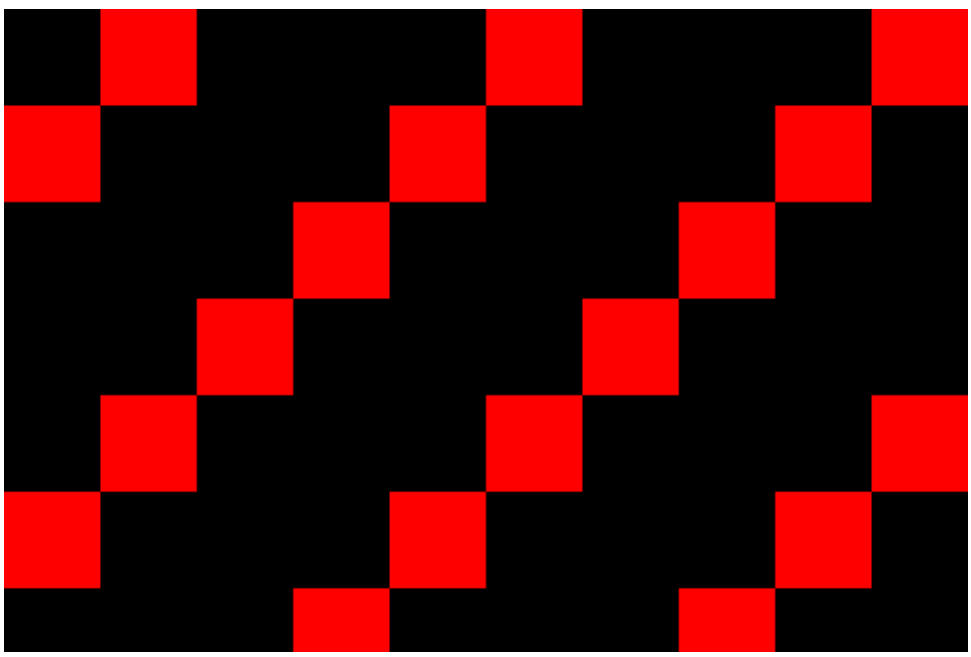
Write the function `onechannel(pattern, rgb)`, which should return a new pattern that contains only the red, green or blue channel of the image.

- rgb = 0, display red
- rgb = 1, display green
- rgb = 2, display blue

For example, running

```
plt.imshow(onechannel(pattern, 0))
```

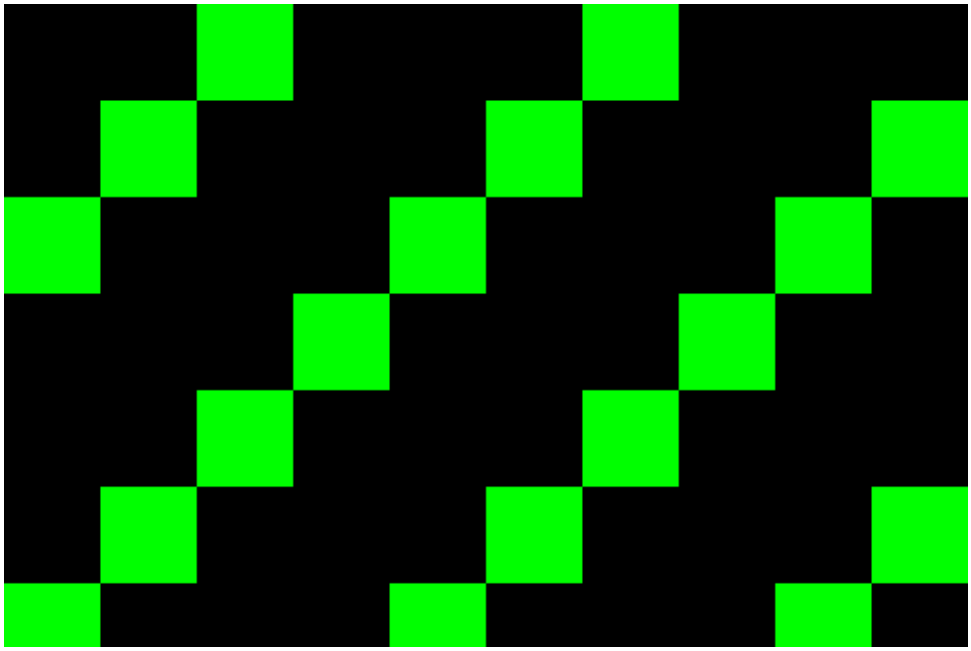
should display:



Running

```
plt.imshow(onechannel(pattern, 1))
```

should display:



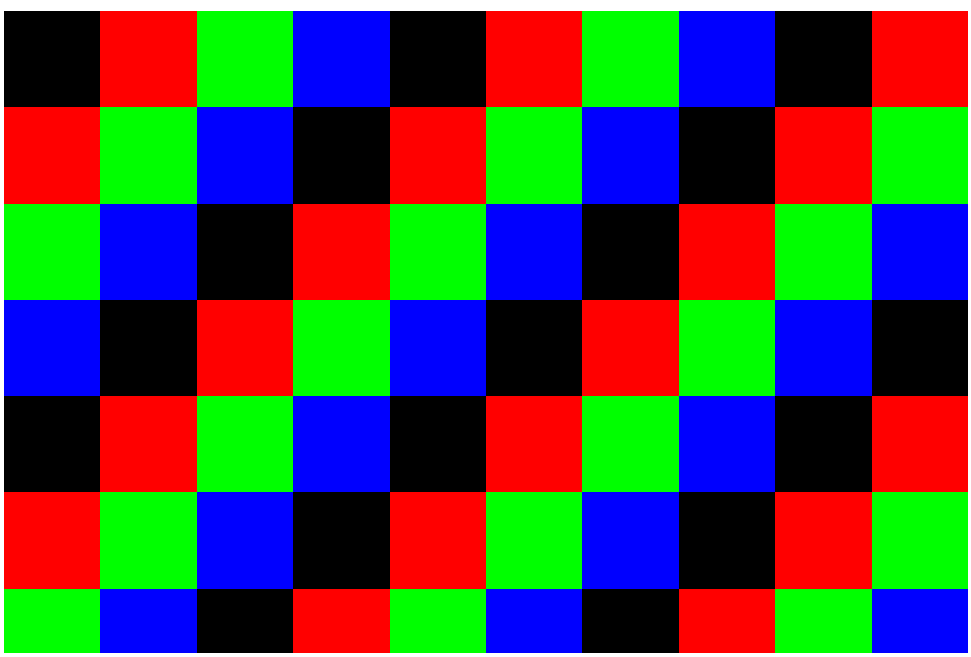
**Part 2 - Channel Permutations (35 points)**

define the function permutecolorchannels(img, perm)

perm is a list of 0,1,2 in some order, which specifies how to permute the color channels. See examples below.

Given the source image 'pattern':

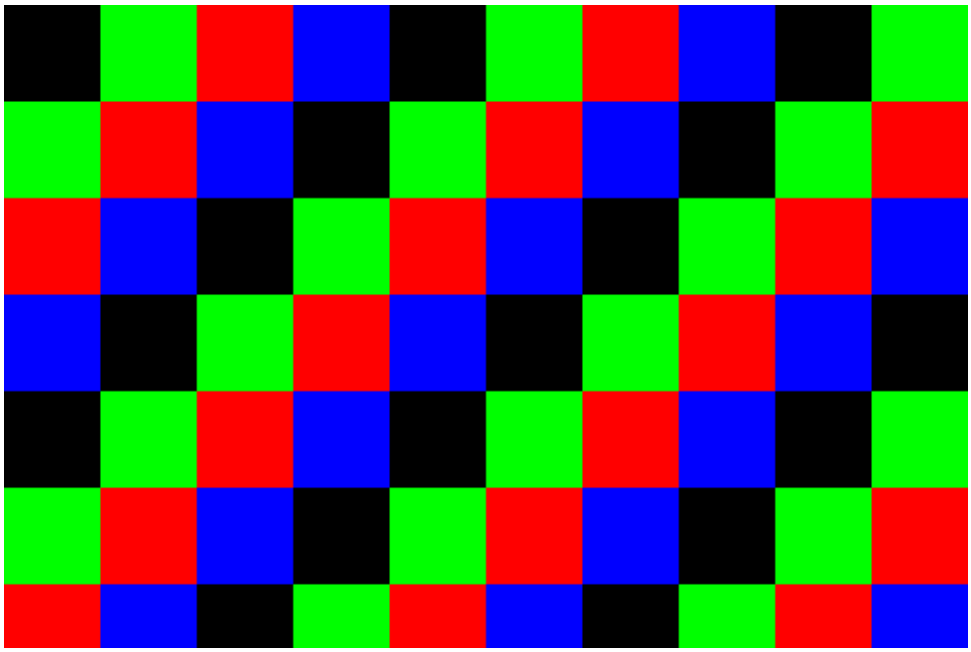
```
plt.imshow(pattern)
```



Running

```
plt.imshow(permutecolorchannels(pattern, [1,0,2]))
```

will produce the following output:



Here is how this works (we will discuss this in detail in class on December 4th):

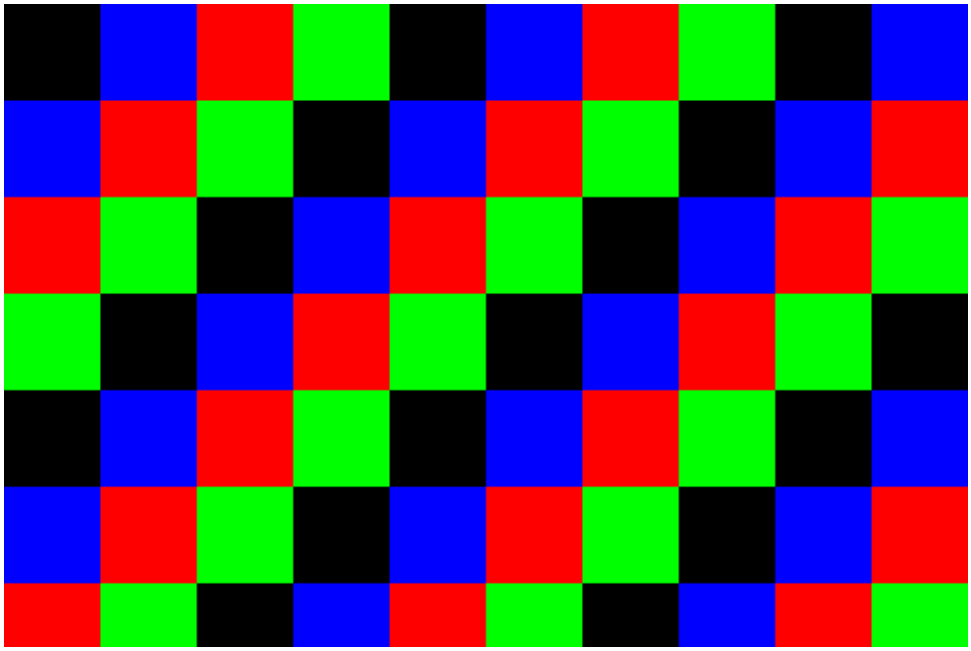
- The first index of the perm list specifies the red channel. So the 1 in that position means that the value of the red color channel should appear in position of the green color channel in the image.
- The second index of the list specifies the green channel. The 0 in that position means that the color of the green color channel should now appear in position of the red color channel.

So this permutation swaps the red and green color channel. (red->green, green->red, blue->blue)

Another example:

```
plt.imshow(permutecolorchannels(pattern, [2,0,1]))
```

should produce this: (red->blue, green->red, blue->green)



Test this function on a real image as follows. First load the file **permcolors.jpg**

```
permcolors = plt.imread('permcolors.jpg')
imageshow(permcolors)
```

which should display:

This image above has its color components permuted.

Use your permutecolorchannels method to restore the correct colors for the image.

```
plt.imshow(permutecolorchannels(permcolors, perm))
```

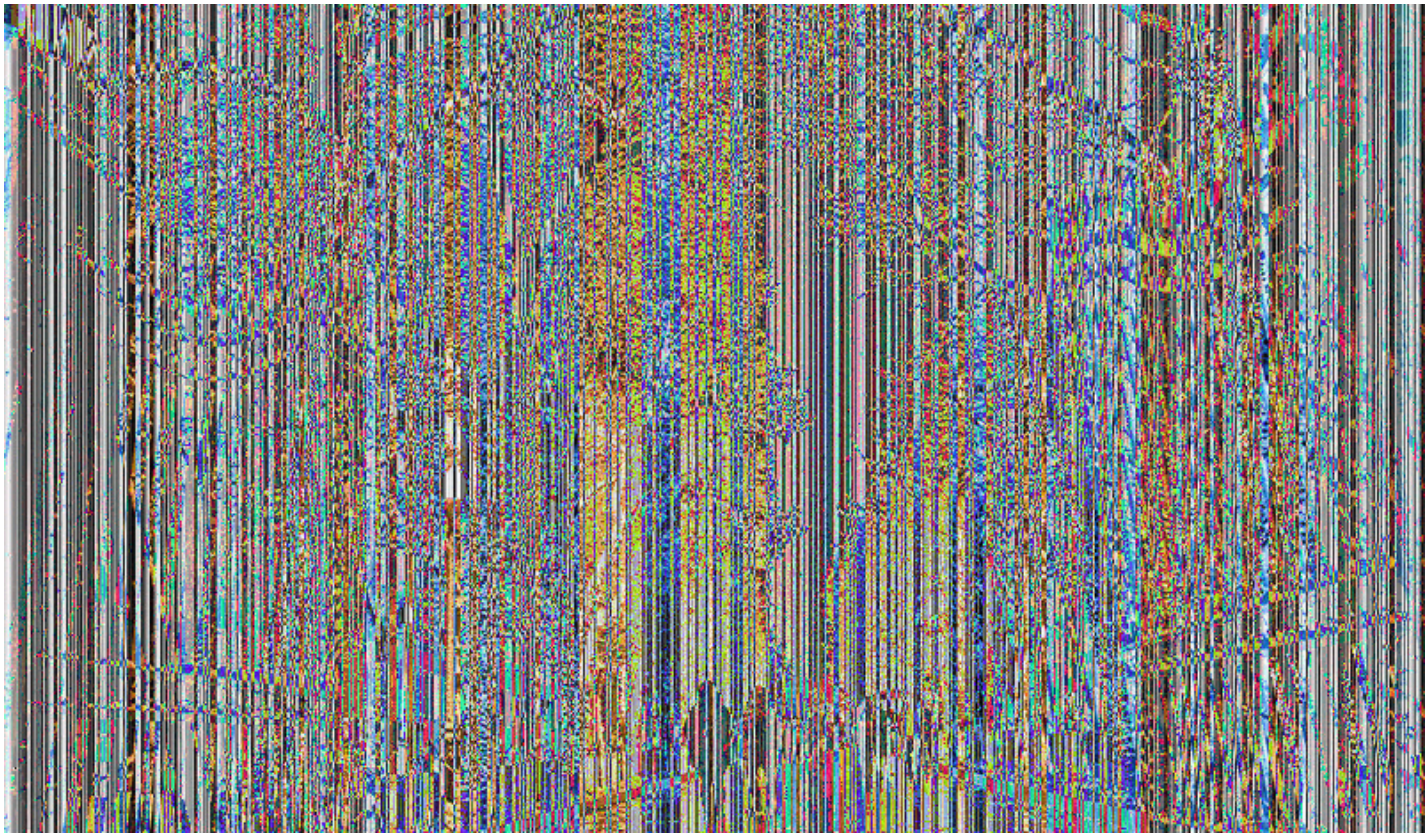
What value of 'perm'  will display the image with correct colors?
Include the correct function call, including the correct permutation,  in your main program.

**Problem 3 - Image encryption with XOR (35 points)**

Running

```
secret = plt.imread('secret.bmp')
plt.imshow(secret)
```

should display this noisy image:



This image has been encrypted by XORing the data with a secret key.  We can recover the original image by XORing the noise image with the key in a certain fashion.

XOR(also known as exclusive OR), is a boolean  'bitwise' operation. You can read about it **here. (https://en.wikipedia.org/wiki/Bitwise_operation#XOR)**

In Python, if A and B are integers, 'A^B' performs a bitwise XOR on the bits representing the integer values of A and B. Numpy arrays also define the '^' operator. The XOR operation has the useful property that A^B^A = B.

We ecncrypt an image by XORing the image with a key. We can recover the image by XORing with the key again.

Download the **key file**. Then load the key (which is a numpy array) using:

```
key = np.load('key.npy')
```

Note that the length of the key is the same as one row of the secret image.

Write the function decrypt(image, key) that takes an image and a key array as a parameter and returns a decrypted image. XOR each red, green, and blue row of the image with the key.

In the main program, call the decrypt function to decrypt secret.bmp and then display the very pretty result.